

Computational Biomathematics: Genetic Algorithms in Biology with AI-Enhanced Coding

Prof. Dr. Dr. h.c. Florian Rupp
Western Caspian University

A genetic algorithm is only as good as the model it implements. Clear thinking about biology and precise communication in prompts are essential for meaningful results.

In this lab, we study genetic algorithms (GAs) as simplified models of biological evolution. The key idea is that a population of individuals evolves over time according to principles inspired by biology: selection, mutation, and recombination. Rather than thinking of a genetic algorithm as a purely technical tool, we will interpret it as a model of how biological traits, strategies, or parameters can evolve under selective pressure.

The general structure of a genetic algorithm is always the same. We start with a population of individuals, evaluate how well each individual performs (its fitness), and then generate a new population by selecting, recombining, and mutating the existing individuals. This process is repeated over many generations.

```
population = initialize_population()

for generation in range(T):
    fitness = evaluate(population)
    selected = selection(population, fitness)
    offspring = crossover(selected)
    population = mutate(offspring)
```

Important Note

Before using AI to generate code, it is essential that you understand what an individual represents in your model, how fitness is defined, and how variation (mutation and crossover) should work.

General Working Strategy

In this lab, you should proceed in the following order. First, carefully understand the biological story of your task. Then translate this story into a computational model by defining individuals, fitness, and update rules. Only after this step should you formulate a prompt for an AI system to generate code. Finally, you must check whether the generated code actually implements your model correctly.

Since the available time is short, you are not expected to build a perfect program from scratch. Instead, your goal is to understand the basic logic of the model, formulate a good prompt, and obtain a small working simulation that you can interpret biologically.

A Small Common Starter Structure

All groups may use the following very small skeleton as a starting point:

```
import numpy as np
import matplotlib.pyplot as plt

def initialize_population(pop_size):
    pass

def fitness(individual):
    pass

def evaluate_population(population):
    return np.array([fitness(ind) for ind in population])

def tournament_selection(population, fitness_values):
    i, j = np.random.randint(0, len(population), size=2)
    return population[i] if fitness_values[i] >= fitness_values[j] else
        population[j]

def mutate(individual, mutation_strength=0.1):
    return individual

T = 50
population_size = 30
```

Hint: You do not need a complicated program. A small and correct implementation is much better than a large and confusing one.

Group A: Evolution of a Trait

We consider a population in which individuals differ in a quantitative trait $x \in [0, 1]$, which could represent, for example, body size or metabolic activity. Biologically, we assume that increasing this trait improves reproductive success, but also introduces a cost. This leads to a natural trade-off between benefit and cost. We model this by the fitness function

$$f(x) = x(1 - x).$$

This function is small when x is very small or very large, and it has a maximum at an intermediate value. Your task is to use a genetic algorithm to discover this optimal trait value.

Student Task

Think carefully about how to represent individuals in your population. The simplest choice is to represent each individual directly as a real number $x \in [0, 1]$. Decide how mutation should modify the trait. A natural idea is to add a small random number and then clip the result back into the interval $[0, 1]$. Implement a genetic algorithm that evolves the population over time and plot how the best fitness and the average fitness change across generations.

Hint: For this group, it is completely sufficient to use real numbers rather than binary strings. This keeps the biology visible and the code simple.

Tiny Starter Snippet

```
def initialize_population(pop_size):
    return np.random.rand(pop_size)

def fitness(x):
    return x * (1 - x)

def mutate(x, mutation_strength=0.05):
    x_new = x + np.random.normal(0, mutation_strength)
    return np.clip(x_new, 0.0, 1.0)
```

AI Prompt Design Task

Write a prompt that asks an AI system to generate a modular Python implementation of this genetic algorithm. Your prompt should clearly describe the trait, the fitness function, and the requirement to visualize the evolution of fitness over time.

Student Task

After running your simulation, explain why the observed optimal trait value makes sense from a biological perspective.

Group B: Parameter Estimation in Population Growth

In this task, you work with a classical biological model of population growth. The logistic equation

$$\dot{x} = rx(1 - x/K)$$

describes how a population grows with rate r and saturates at a carrying capacity K . In practice, these parameters are often unknown and must be estimated from data. Your goal is to use a genetic algorithm to find values of r and K that best reproduce observed data.

Student Task

Represent each individual as a pair (r, K) . Define a fitness function that measures how well the model fits the data, for example by computing the squared error between simulated and observed values. Implement a genetic algorithm that evolves these parameters over time. Visualize both the best-fit curve and the observed data.

Hint: Do not worry about solving the differential equation exactly. A simple discrete-time approximation is enough for this lab. The main point is that the genetic algorithm searches for parameter values that produce a good fit.

Synthetic Data for the Logistic Growth Task

For this task, use the following synthetic population data:

t	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$x_{\text{data}}(t)$	6.2	8.1	10.7	14.6	18.9	24.7	31.5	39.8	48.6	56.9	64.1	69.5	74.0	77.1	79.3

These data are assumed to come from a logistic growth process of the form

$$\dot{x} = rx \left(1 - \frac{x}{K}\right),$$

where the parameters r and K are unknown. Your task is to use a genetic algorithm to estimate suitable values of r and K so that the model fits the data as well as possible. A simple fitness function is based on the squared fitting error:

$$\text{error}(r, K) = \sum_{i=1}^n (x_{\text{model}}(t_i) - x_{\text{data}}(t_i))^2.$$

Since genetic algorithms often maximize fitness, you may define

$$\text{fitness}(r, K) = -\text{error}(r, K).$$

Hint: Assume the bounds $0.1 \leq r \leq 1.5$ as well as $20 \leq K \leq 120$.

Tiny Starter Snippet

```
def simulate_logistic(r, K, x0=0.2, T=20, dt=0.1):
    x = x0
    values = [x]
    for _ in range(T - 1):
        x = x + dt * r * x * (1 - x / K)
        values.append(x)
    return np.array(values)

data = np.array([0.2, 0.23, 0.27, 0.32, 0.38, 0.45, 0.53, 0.60,
                 0.66, 0.71, 0.75, 0.78, 0.80, 0.82, 0.83, 0.84,
                 0.85, 0.85, 0.86, 0.86])

def initialize_population(pop_size):
    r_values = np.random.uniform(0.1, 2.0, pop_size)
```

```

K_values = np.random.uniform(0.5, 2.0, pop_size)
return np.column_stack((r_values, K_values))

def fitness(individual):
    r, K = individual
    model = simulate_logistic(r, K, x0=data[0], T=len(data))
    error = np.sum((model - data)**2)
    return -error

```

AI Prompt Design Task

Write a prompt asking an AI system to generate a modular program that performs parameter estimation using a genetic algorithm. Make sure to include simulation of the model and visualization of the fit.

Student Task

Investigate how the mutation rate affects the quality of the parameter estimation. What happens if mutation is too large or too small?

Group C: Evolution of Antibiotic Resistance

In this scenario, we study a population of bacteria exposed to antibiotics. Each bacterium is characterized by a resistance level $x \in [0, 1]$. Higher resistance increases survival in the presence of antibiotics, but comes at a cost, for example due to metabolic burden.

We model this trade-off using a parameter $c > 0$, which measures how costly resistance is. The fitness function depends on the environment:

$$f(x) = \begin{cases} 1 - x, & \text{no antibiotic present,} \\ x(1 - cx), & \text{antibiotic present.} \end{cases}$$

Thus, in the absence of antibiotics, resistance is purely costly, whereas in the presence of antibiotics it becomes beneficial but still incurs a cost.

Student Task

Implement a genetic algorithm that evolves the resistance level under two different environments: one without antibiotics and one with antibiotics. Compare how the distribution of resistance evolves in both cases. Visualize the evolution of fitness over time.

Hint: The constant c measures the biological cost of resistance. If c is large, resistance is expensive. If c is small, resistance is relatively cheap.

Tiny Starter Snippet

```

def initialize_population(pop_size):
    return np.random.rand(pop_size)

```

```
def fitness_no_antibiotic(x):
    return 1 - x

def fitness_with_antibiotic(x, c=0.8):
    return x * (1 - c * x)

def mutate(x, mutation_strength=0.05):
    x_new = x + np.random.normal(0, mutation_strength)
    return np.clip(x_new, 0.0, 1.0)
```

AI Prompt Design Task

Write a prompt asking an AI system to implement a genetic algorithm with environment-dependent fitness. Make sure that the prompt explicitly requests comparison between the two scenarios.

Student Task

Explain how the parameter c influences the outcome. What happens if resistance is very costly? What happens if it is cheap?

Group D: Evolution of Cooperation

In this task, individuals repeatedly interact with each other and decide whether to cooperate or defect. Each individual is characterized by a probability $p \in [0, 1]$ of cooperating. Thus, $p = 1$ means an individual always cooperates, $p = 0$ means it always defects, and intermediate values correspond to mixed behavior. The success of an individual depends on the outcomes of these interactions. Over time, strategies evolve depending on how successful they are.

This group project is conceptually a little harder than the others, so it is important to keep the model simple. You do *not* need a sophisticated game-theoretic model. A small toy version is enough. A very simple choice is the following. When two individuals with cooperation probabilities p_i and p_j meet, we compute their expected payoff from a Prisoner's Dilemma style interaction. For example, one may use the payoff values

$$R = 3, \quad T = 5, \quad S = 0, \quad P = 1,$$

where R is the reward for mutual cooperation, T is the temptation payoff, S is the sucker payoff, and P is the punishment for mutual defection.

Using probabilities instead of deterministic choices, the expected payoff of individual i against individual j can be written as

$$p_i p_j R + p_i (1 - p_j) S + (1 - p_i) p_j T + (1 - p_i) (1 - p_j) P.$$

You may then define the fitness of an individual as the average payoff it receives when interacting with several other individuals in the population.

Student Task

Keep the model simple. Represent each individual by a single number $p \in [0,1]$, its probability of cooperating. Define a function that computes the expected payoff of one individual against another. Then compute the fitness of an individual by averaging these payoffs against the rest of the population, or against a small random sample. Implement a genetic algorithm that evolves the cooperation probability over time and track how the average level of cooperation changes.

Hints:

- You do not need repeated memory, strategy trees, or complicated adaptive rules. A single cooperation probability p is enough for this lab. The goal is to see whether selection tends to push the population toward more cooperation or more defection.
- If computing all pairwise interactions feels too heavy, let each individual interact with only a few randomly chosen partners. That is perfectly acceptable here.

Tiny Starter Snippet

```
R, Tpay, S, P = 3.0, 5.0, 0.0, 1.0

def initialize_population(pop_size):
    return np.random.rand(pop_size)

def expected_payoff(p_i, p_j):
    return (p_i * p_j * R
            + p_i * (1 - p_j) * S
            + (1 - p_i) * p_j * Tpay
            + (1 - p_i) * (1 - p_j) * P)

def fitness_of_individual(population, i):
    payoffs = []
    for j in range(len(population)):
        if i != j:
            payoffs.append(expected_payoff(population[i], population[j]))
    return np.mean(payoffs)

def evaluate_population(population):
    return np.array([fitness_of_individual(population, i)
                     for i in range(len(population))])

def mutate(p, mutation_strength=0.05):
    p_new = p + np.random.normal(0, mutation_strength)
    return np.clip(p_new, 0.0, 1.0)
```

AI Prompt Design Task

Write a prompt asking an AI system to implement this model, including pairwise interactions, fitness calculation from expected payoffs, and evolutionary updates of the cooperation probability.

Student Task

Discuss under which conditions cooperation emerges. How does mutation influence the outcome?

A Simple Selection and Reproduction Hint for Everyone

If you are unsure how to generate a new population, you may use a very simple method. Repeatedly select two parents by tournament selection, average them to form a child, and then mutate the child.

```
def make_child(parent1, parent2):
    return 0.5 * (parent1 + parent2)

def next_generation(population, fitness_values, mutation_strength=0.05):
    new_pop = []
    for _ in range(len(population)):
        p1 = tournament_selection(population, fitness_values)
        p2 = tournament_selection(population, fitness_values)
        child = make_child(p1, p2)
        child = mutate(child, mutation_strength)
        new_pop.append(child)
    return np.array(new_pop)
```

Hint: For Group B, where individuals are parameter pairs (r, K) , the same idea works componentwise: average the two parent vectors and then mutate each coordinate slightly.

Reflection**Student Task**

Reflect on your work. Was it easy to describe your model precisely in a prompt? Did the AI-generated code match your expectations? What difficulties did you encounter, and how could you improve your prompt?

Final Presentations

Each group presents the biological problem, the way individuals were represented, their best prompt, one simulation result, and one key insight. Since time is short, focus on communicating the main idea clearly rather than showing many technical details.